

Introduction aux CNNs



MonCoachData



Deep Learning

- Bienvenue dans la section sur les réseaux de neurones convolutifs !
- Les CNNs (pour Convolutional Neural Networks) sont une architecture spécifique des réseaux de neurones qui sont extrêmement efficaces pour traiter les données d'images.
- Passons en revue ce que nous allons apprendre dans cette section !





Deep Learning

- Section CNN
 - Comprendre les CNN
 - Noyaux et filtres d'image
 - Convolutions
 - Couches de Pooling
 - Dataset MNIST (Images en niveaux de gris)
 - Dataset CIFAR-10 (Images en couleur)





Deep Learning

- Section CNN
 - Travailler avec des fichiers d'images (jpg, png, etc...) avec les CNNs.
 - CNNs pour la classification des cellules sanguines du paludisme (ou malaria).
 - Exercice CNN sur le Fashion Image Dataset.



Filtres d'Images





Deep Learning

- Avant de nous plonger dans les CNNs, discutons d'abord de quelques idées clés en matière de vision par ordinateur.
- La vision par ordinateur (ou Computer Vision) est un terme général qui désigne l'utilisation de programmes informatiques pour traiter des données d'images.





Deep Learning

- Si vous avez déjà utilisé un logiciel de retouche photo, vous avez probablement vu des filtres, comme un filtre de flou. Mais comment fonctionnent-ils ?





Deep Learning

- Les filtres sont en fait un **noyau d'image**, qui est une petite matrice appliquée à une image entière.
- Certains filtres populaires sont bien connus, par exemple un filtre flou :

$$\begin{pmatrix} 0.0625 & 0.125 & 0.0625 \\ 0.125 & 0.25 & 0.125 \\ 0.0625 & 0.125 & 0.0625 \end{pmatrix}$$



Deep Learning

- Voyons comment ces noyaux/filtres d'image sont réellement appliqués à une image.





Deep Learning

- Les filtres nous permettent de transformer les images

0	0	0	0	0	0
0	1	1	1	1	0
0	1	-1	-1	1	0
0	1	-1	-1	1	0
0	1	1	1	1	0
0	0	0	0	0	0



Deep Learning

- Ici, nous avons une image en niveaux de gris

0	0	0	0	0	0
0	1	1	1	1	0
0	1	-1	-1	1	0
0	1	-1	-1	1	0
0	1	1	1	1	0
0	0	0	0	0	0





Deep Learning

- Valeurs échelonnées entre -1 et 1

0	0	0	0	0	0
0	1	1	1	1	0
0	1	-1	-1	1	0
0	1	-1	-1	1	0
0	1	1	1	1	0
0	0	0	0	0	0





Deep Learning

- Les filtres nous permettent de transformer les images

0	0	0	0	0	0
0	1	1	1	1	0
0	1	-1	-1	1	0
0	1	-1	-1	1	0
0	1	1	1	1	0
0	0	0	0	0	0

0	0	0
0	-1	1
0	1	-1

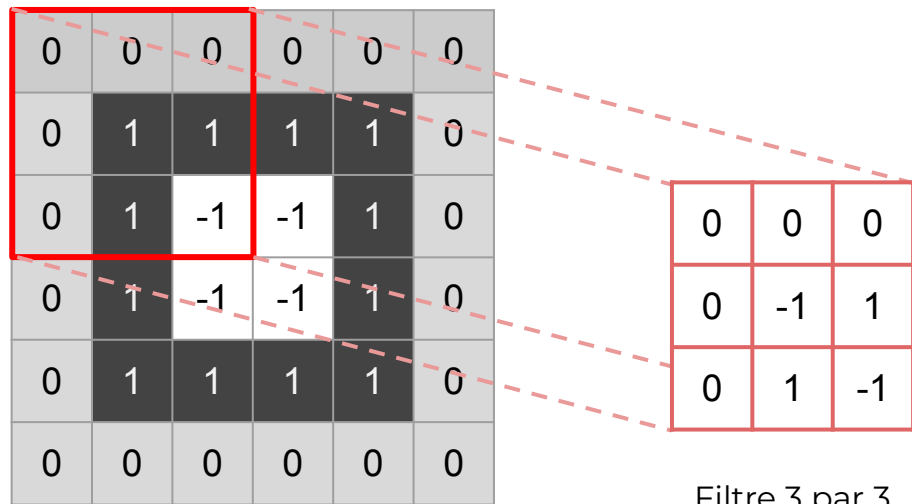
Filtre 3 par 3





Deep Learning

- Les filtres nous permettent de transformer les images



Filtre 3 par 3



Deep Learning

- Les filtres sont essentiellement une matrice



Filtre 3 par 3

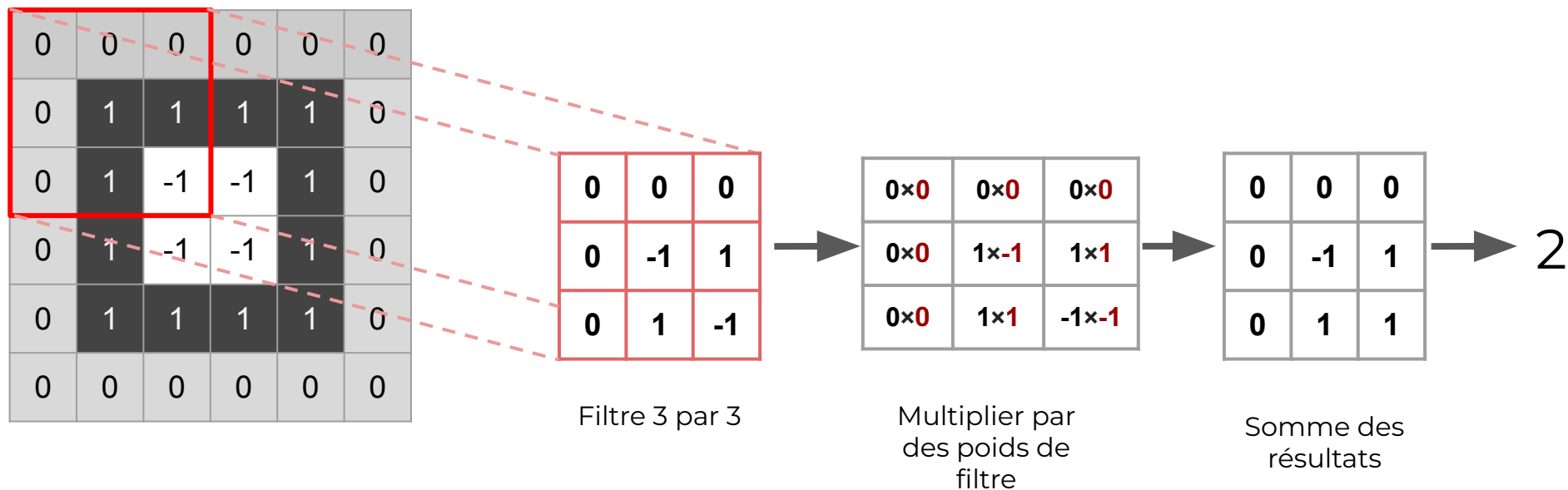
Multiplier par
les poids du
filtre





Deep Learning

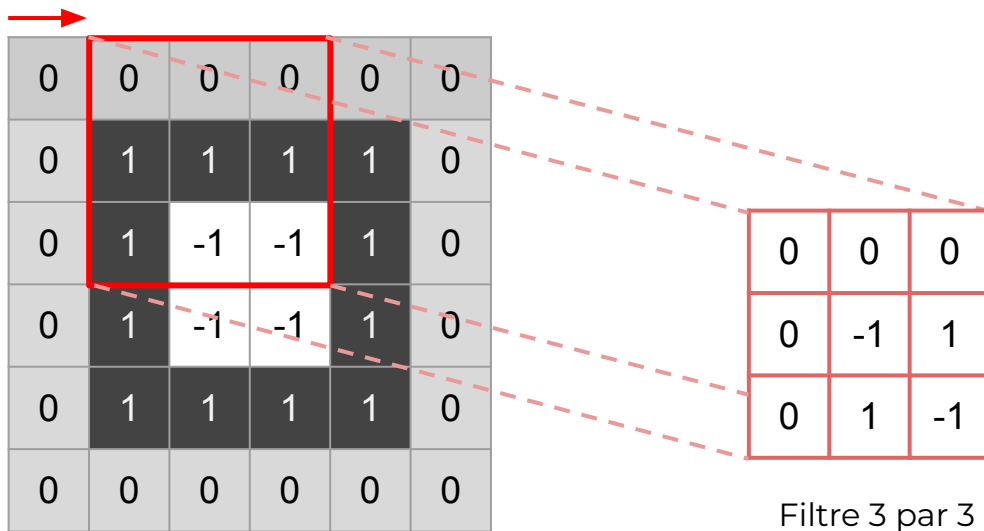
- Remarquez comment la résolution va diminuer





Deep Learning

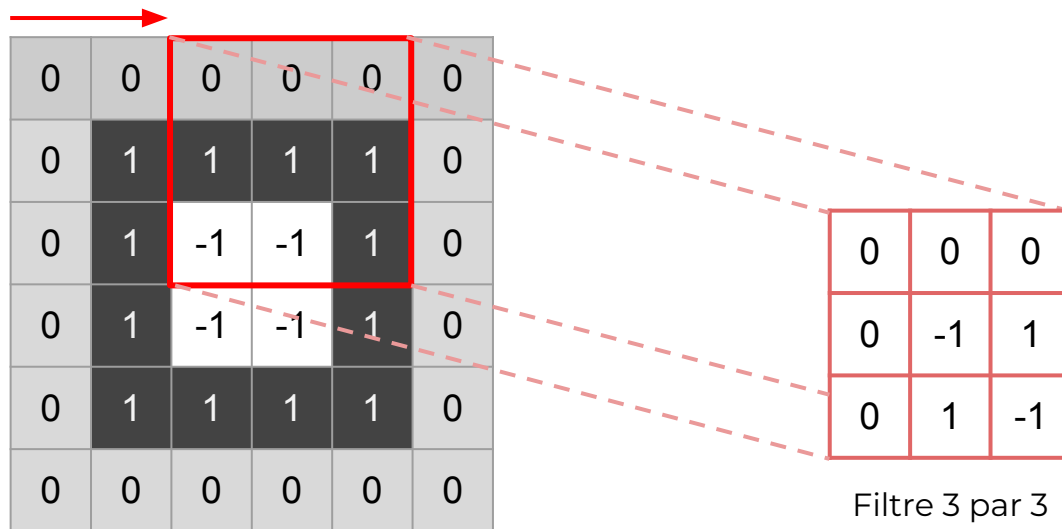
- Nous pouvons également modifier notre distance de pas (stride)





Deep Learning

- Exemple de distance stride de 2





Deep Learning

- Voici un exemple interactif !
 - **setosa.io/ev/image-kernels/**





Deep Learning

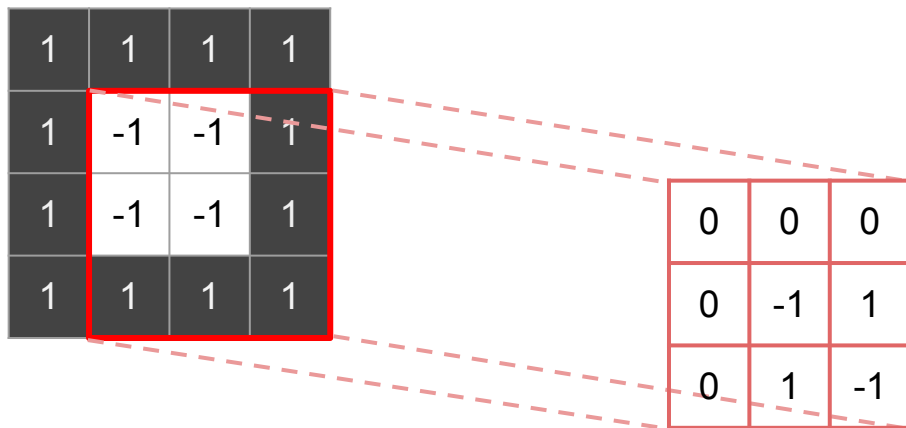
- Dans le contexte des CNNs, ces "filtres" sont appelés **noyaux de convolution**.
- Le processus qui consiste à les faire passer sur une image est connu sous le nom de **convolution**.
- Passons en revue quelques autres facteurs importants !





Deep Learning

Pendant la convolution, nous pourrions perdre les frontières



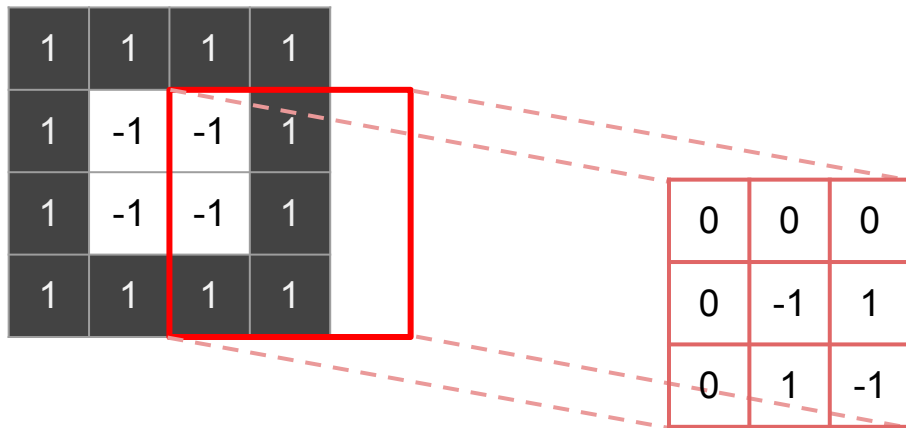
Filtre 3 par 3





Deep Learning

Pendant la convolution, nous pourrions perdre les frontières



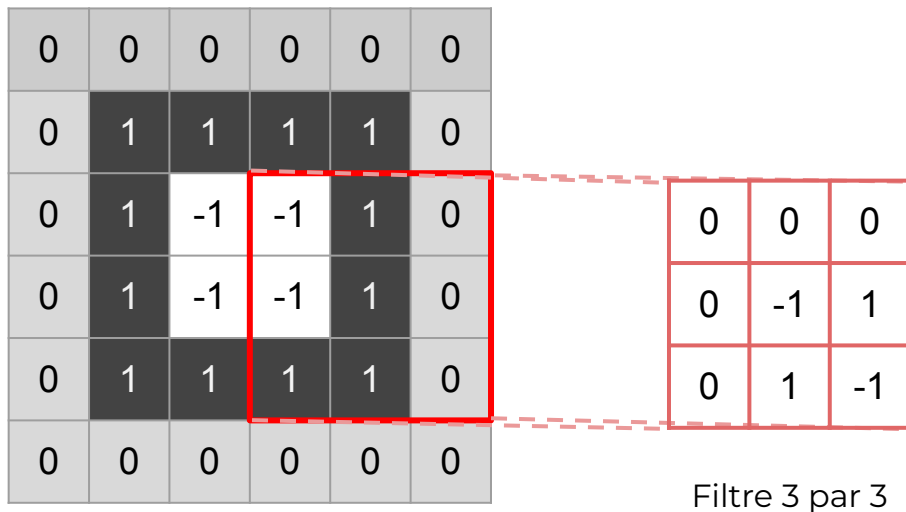
Filtre 3 par 3





Deep Learning

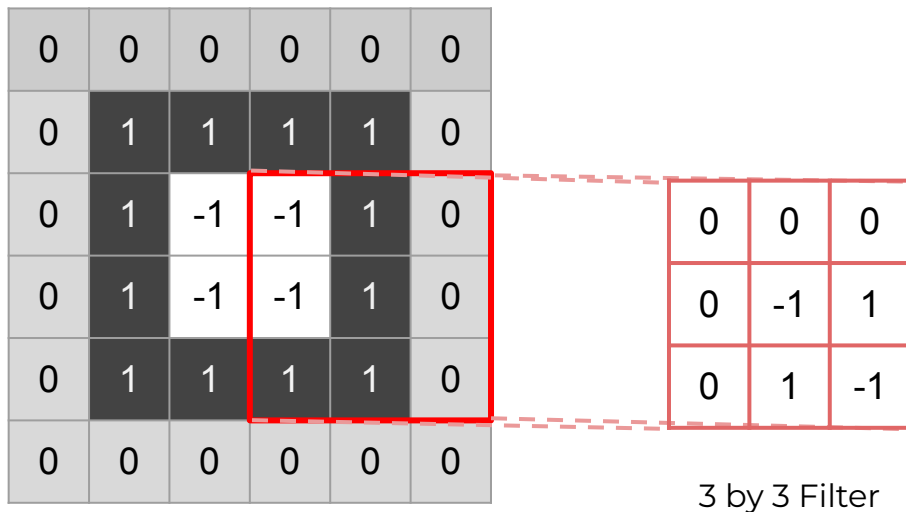
On peut **rembourrer** l'image avec plus de valeurs





Deep Learning

Cela nous permet de préserver la taille de l'image





Deep Learning

- Maintenant que nous comprenons les filtres d'image, explorons l'architecture d'un CNN qui permet au réseau de trouver les meilleurs poids pour un filtre dans la **couche de convolution**.



Couches de Convolution



MonCoachData



Deep Learning

- Rappelons que l'utilisation d'un ANN pour l'ensemble des données du MNIST permet d'obtenir un réseau d'une exactitude relativement bonne.
- Cependant, le fait de toujours utiliser les modèles ANN pour les données d'images pose certains problèmes.





Deep Learning

- ANNs
 - Grande quantité de paramètres (plus de 100 000 pour des images minuscules de 28 par 28)
 - Nous perdons toutes les informations en 2D en aplatissant l'image
 - Ne fonctionnera que sur des images très similaires et bien centrées.





Deep Learning

- Un CNN peut utiliser des couches de convolution pour aider à atténuer ces problèmes.
- Une couche de convolution est créée lorsque nous appliquons plusieurs filtres d'image aux images d'entrée.
- La couche sera ensuite entraînée à déterminer les meilleures valeurs de poids de filtre.





Deep Learning

- Un CNN permet également de réduire les paramètres en se concentrant sur la **connectivité locale**.
- Tous les neurones ne seront pas entièrement connectés.
- Au lieu de cela, les neurones sont seulement connectés à un sous-ensemble de neurones locaux dans la couche suivante (ceux-ci finissent par être les filtres !)





Deep Learning

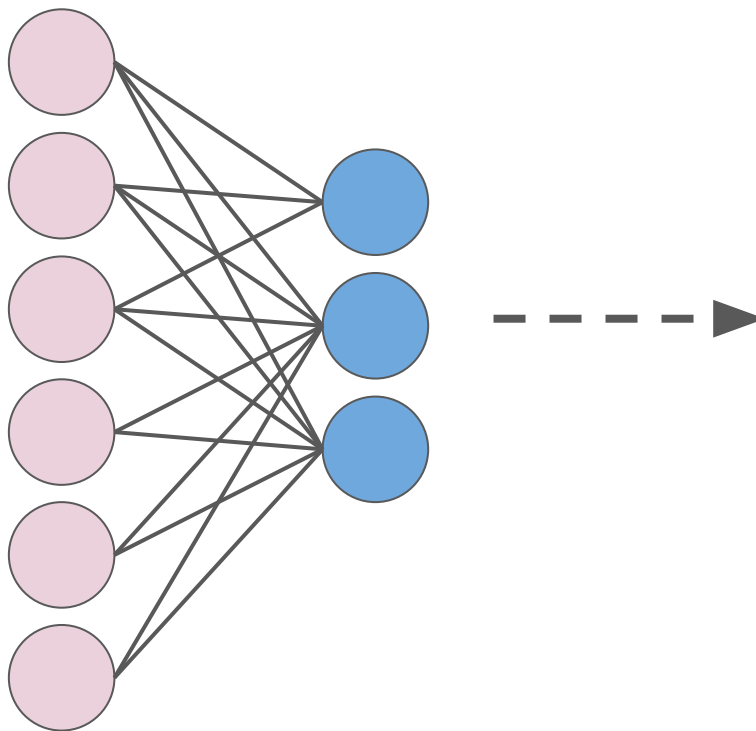
- Comprendons cette connectivité locale et sa connexion aux filtres en commençant par un exemple 1-D simplifié.
- Nous étendrons plus tard ce système à des entrées 2-D pour une image en niveaux de gris, puis plus tard à des entrées 3-D du tenseur pour des images en couleur.





Deep Learning

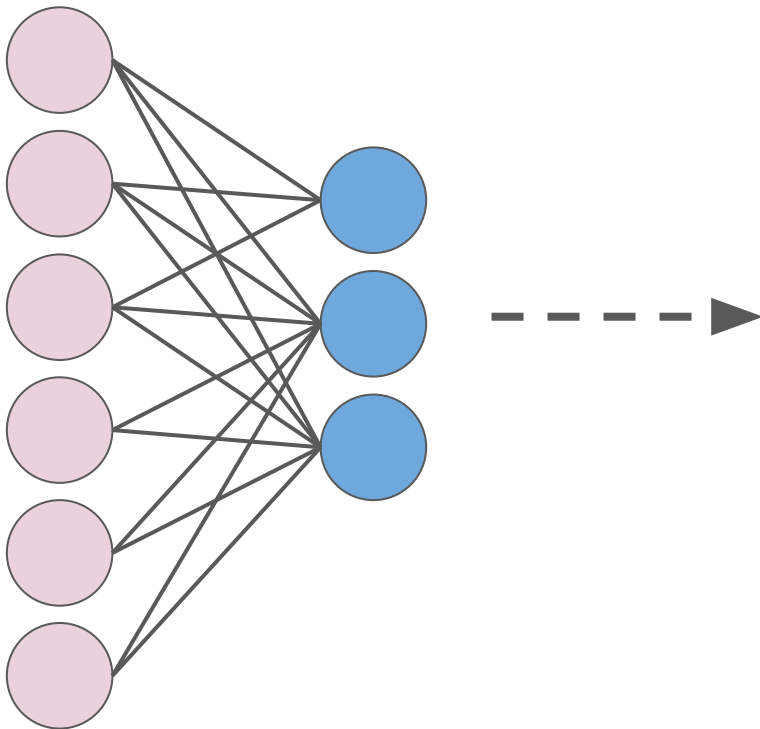
- ANN





Deep Learning

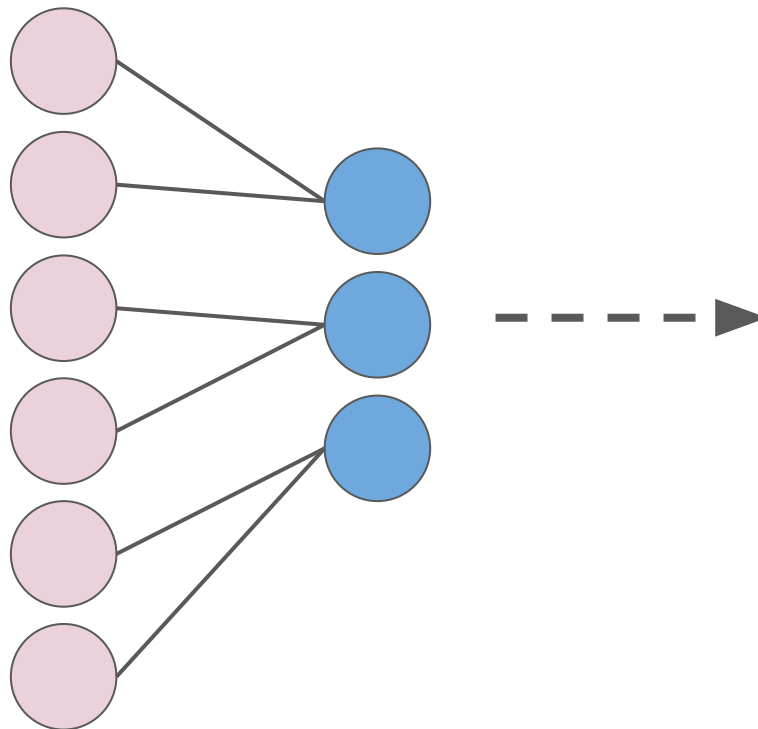
- Entièrement connectés, beaucoup de paramètres !





Deep Learning

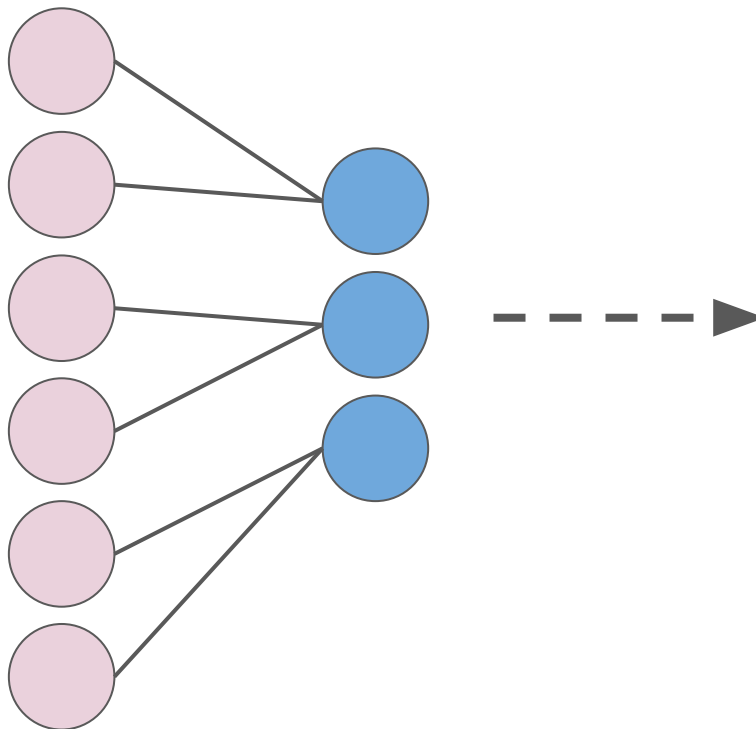
- CNN - Couche de Convolution





Deep Learning

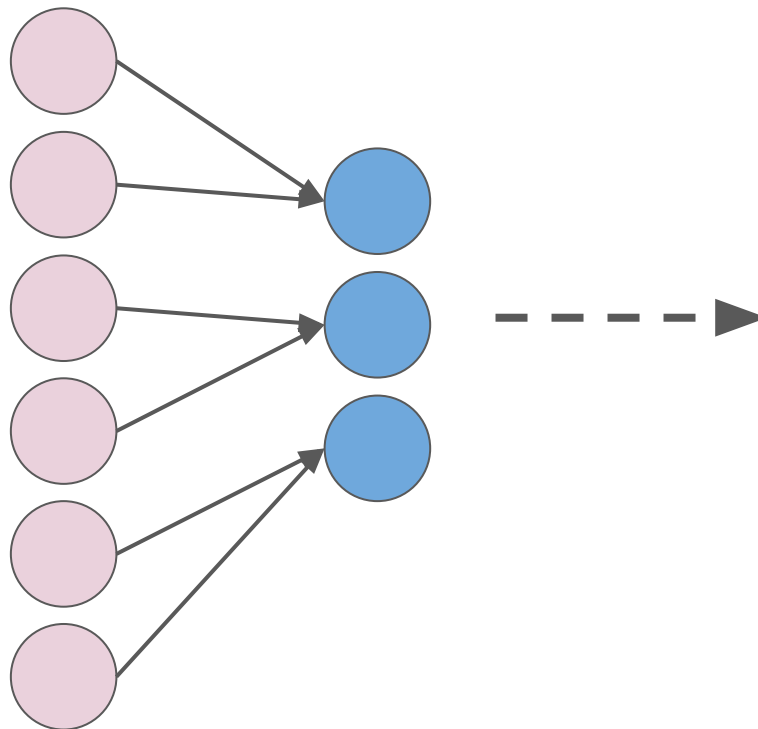
- CNN - Connexions localisées





Deep Learning

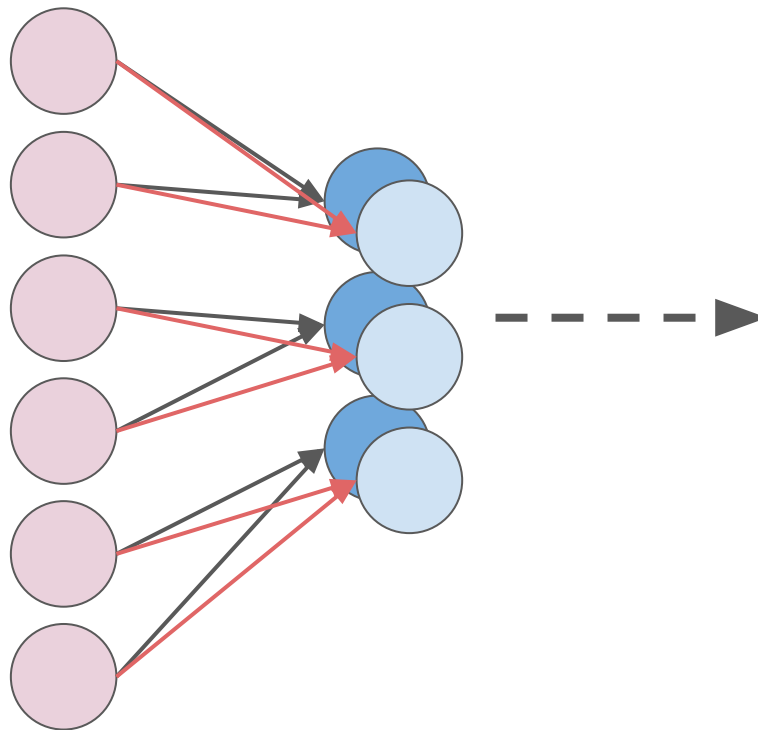
- CNN - Nous avons ici 1 filtre





Deep Learning

- CNN - Nous avons ici 2 filtres





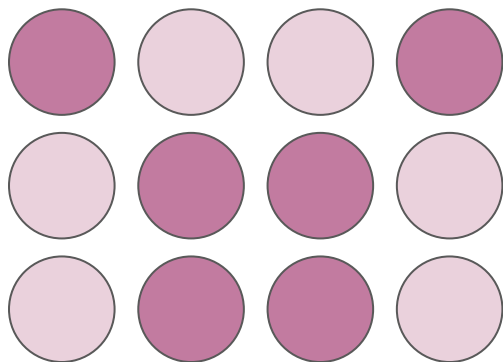
Deep Learning

- Nous venons de voir un exemple de convolution en 1-D, mais souvenez-vous, les images en niveaux de gris sont en 2-D !
- De plus, nous voulons préserver ces informations relationnelles en 2-D dans la couche de convolution.





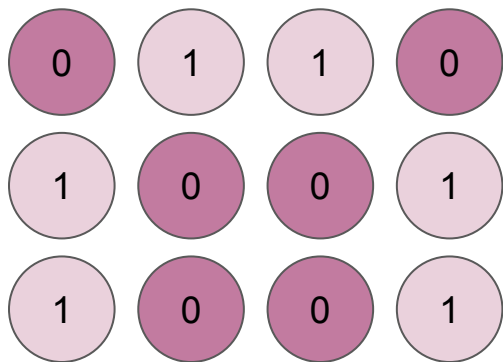
Deep Learning



Input Image



Deep Learning

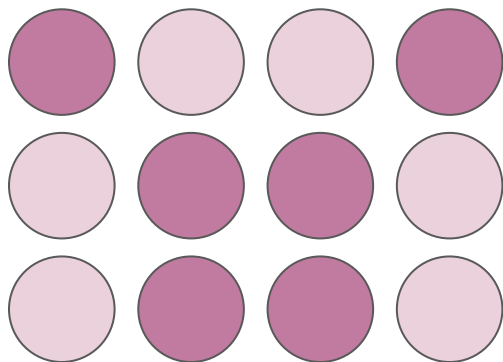


Input Image





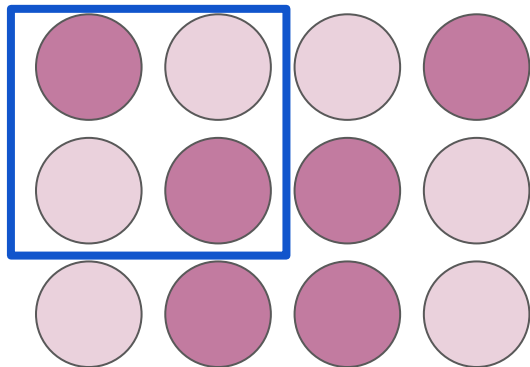
Deep Learning



Input Image



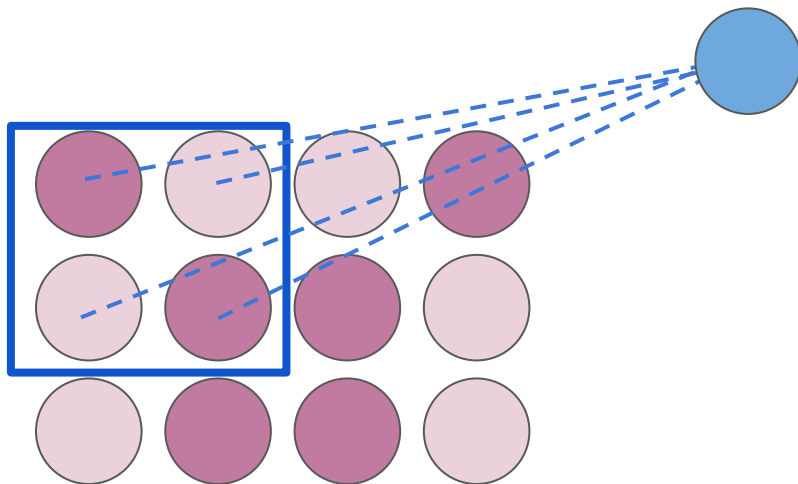
Deep Learning



Input Image



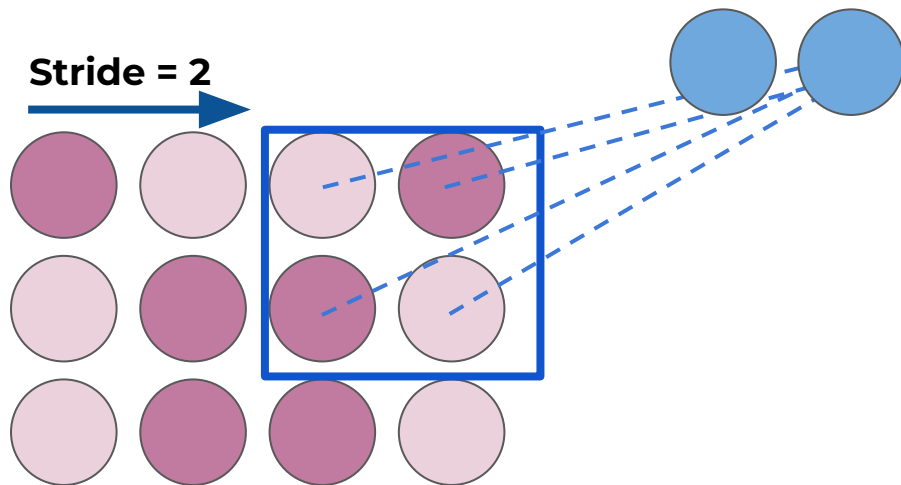
Deep Learning



Input Image



Deep Learning

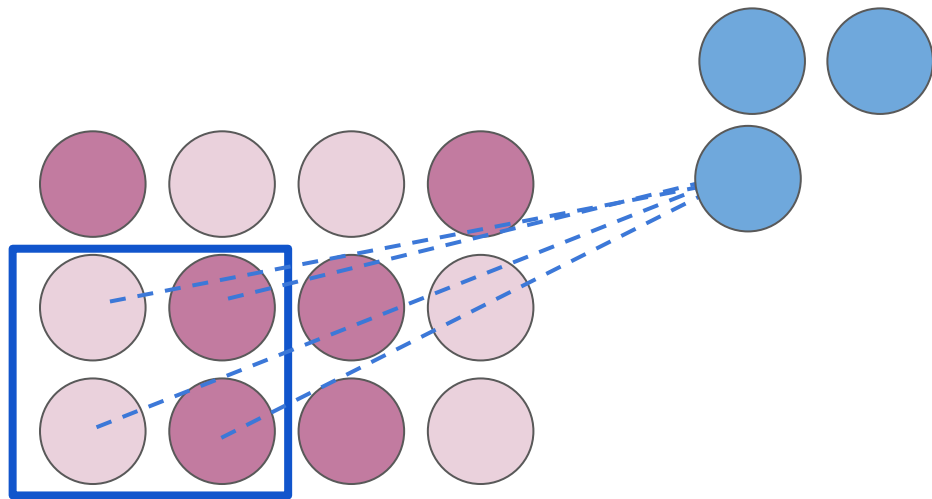


Input Image





Deep Learning

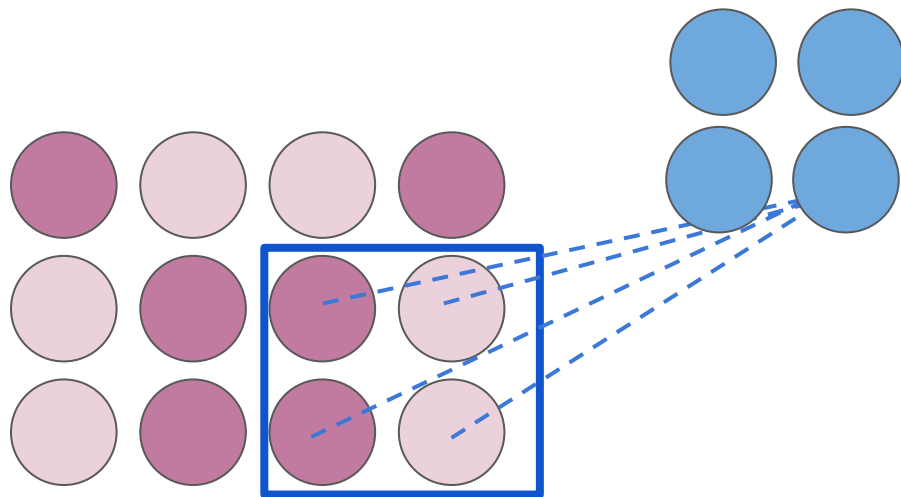


Input Image





Deep Learning

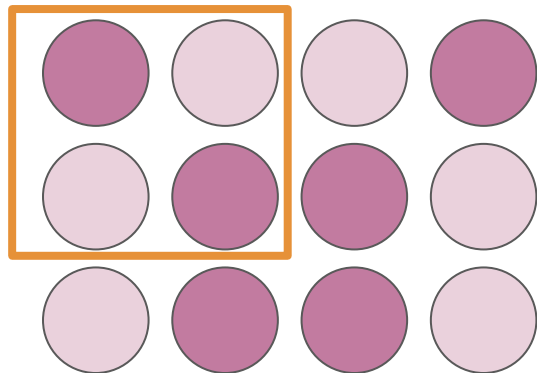


Input Image

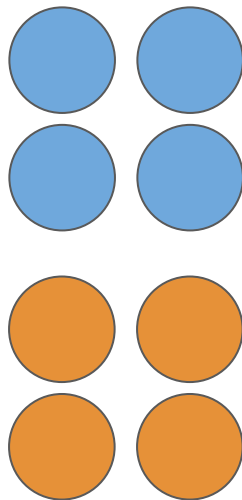




Deep Learning

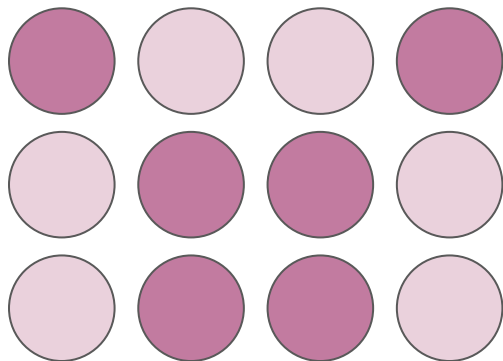


Input Image

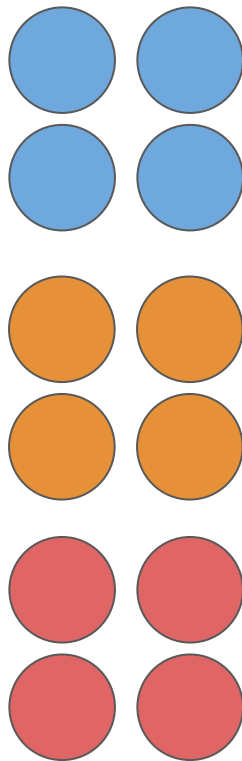




Deep Learning

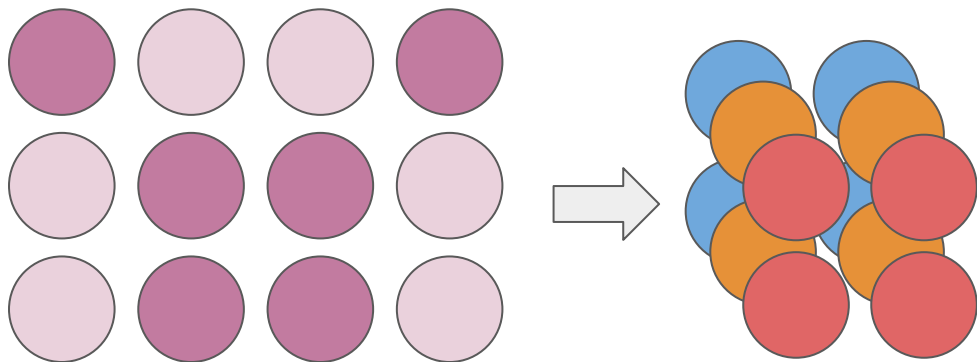


Input Image





Deep Learning



Input Image

**Convolutional
Layer**





Deep Learning

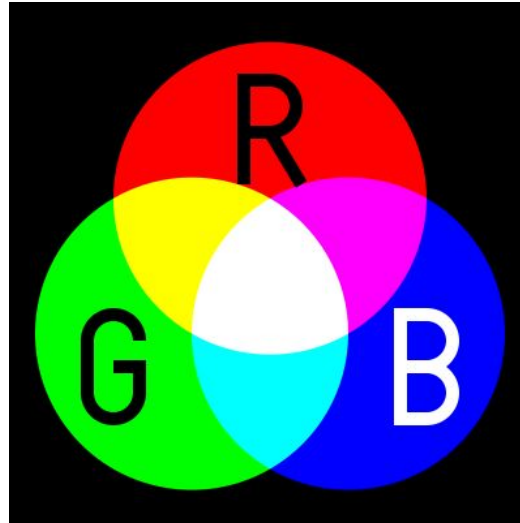
- Mais c'était juste en 2D pour les images en niveaux de gris.
- Qu'en est-il des images en couleur ?
- Les images couleur peuvent être considérées comme des tenseurs 3D composés de canaux de couleur rouge, verte et bleue.





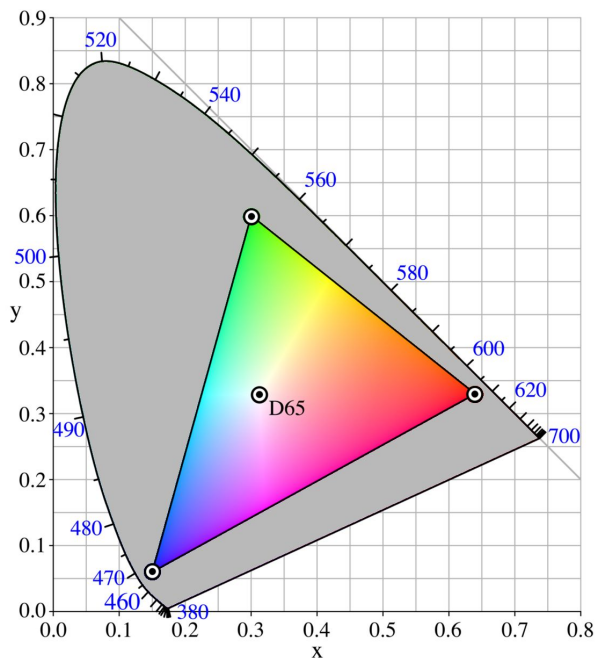
Deep Learning

- Le mélange de couleurs additives nous permet de représenter une grande variété de couleurs en combinant simplement différentes quantités ou R, G, B.





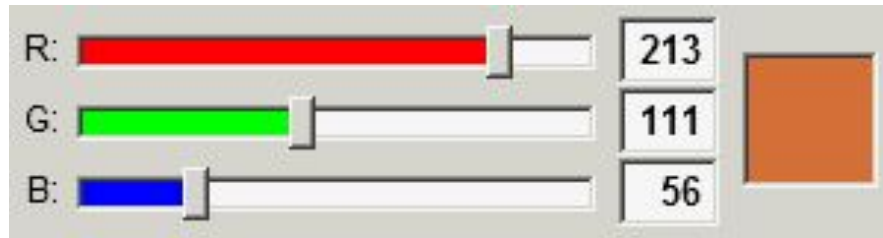
- RGB permet de produire une gamme de couleurs





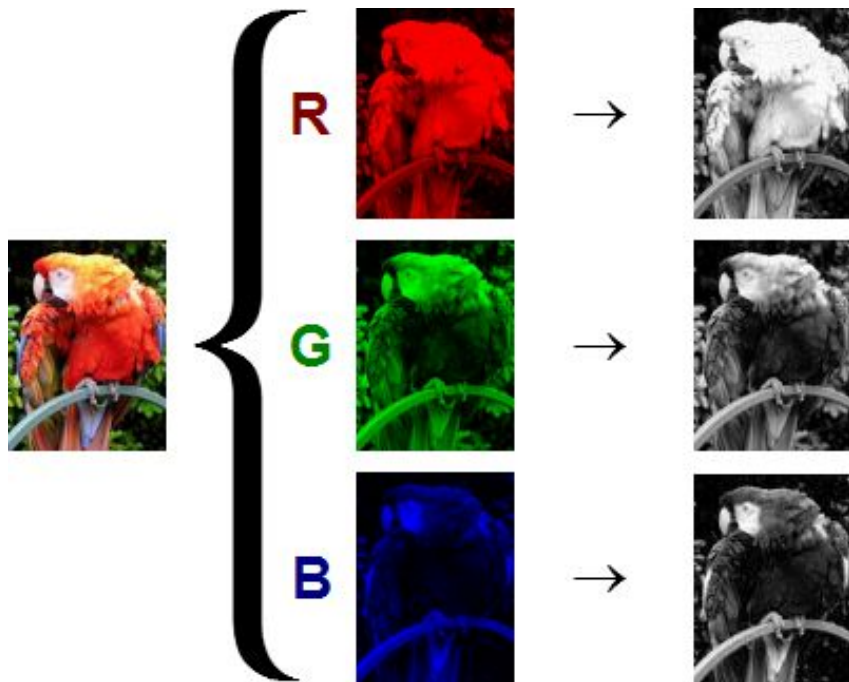
Deep Learning

- Chaque canal de couleur aura des valeurs d'intensité.
- Vous avez peut-être déjà vu ce type de représentation dans d'autres logiciels avec des curseurs RGB.





Deep Learning



- La forme de la matrice de couleurs a alors 3 dimensions.
- Hauteur (Height)
- Largeur (Width)
- Canaux de couleur





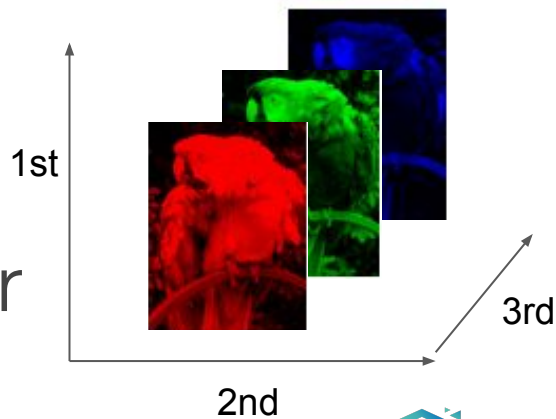
- Cela signifie que lorsque vous lisez dans une image et que vous vérifiez sa forme, elle ressemble à quelque chose :
 - **(1280,720,3)**
 - **1280** pixel de large
 - **720** pixel de haut
 - **3** canaux de couleur





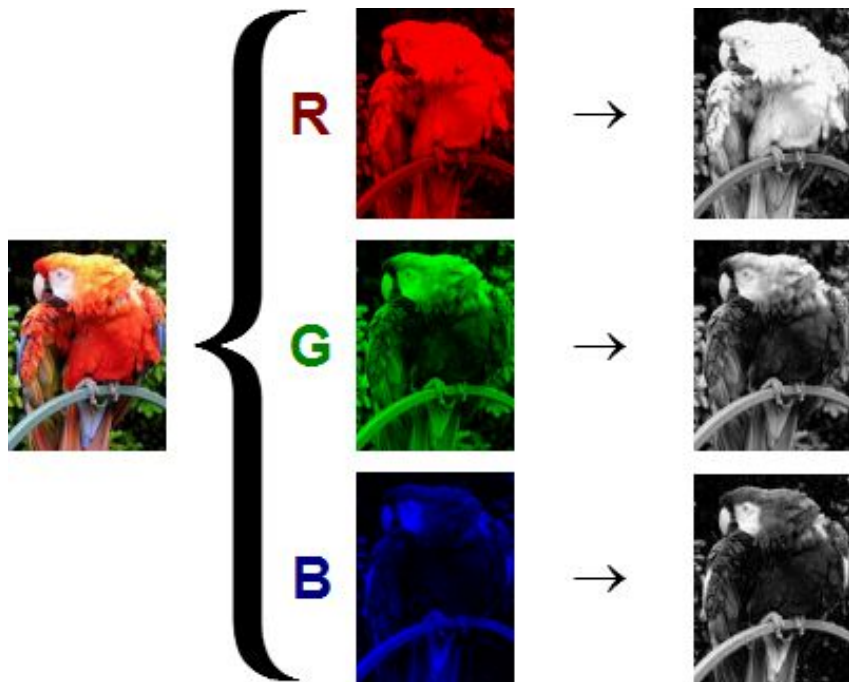
Deep Learning

- Cela signifie que lorsque vous lisez dans une image et que vous vérifiez sa forme, elle ressemble à quelque chose :
 - **(720,1280,3)**
 - **720** pixel de large
 - **1280** pixel de haut
 - **3** canaux de couleur





Deep Learning



- Gardez à l'esprit que l'ordinateur ne "saura" pas qu'un canal est rouge, il sait juste qu'il y a maintenant 3 canaux d'intensité.





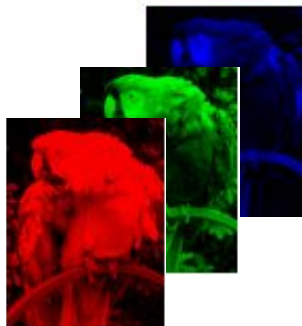
Deep Learning

- Comment effectuer ensuite une convolution sur une image couleur ?
- On se retrouve avec un filtre 3D, avec des valeurs pour chaque canal de couleur.

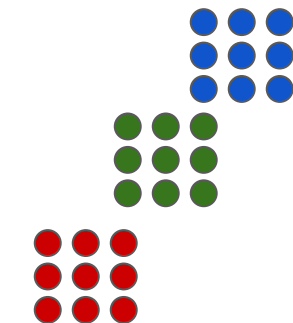




Deep Learning



Input Layer

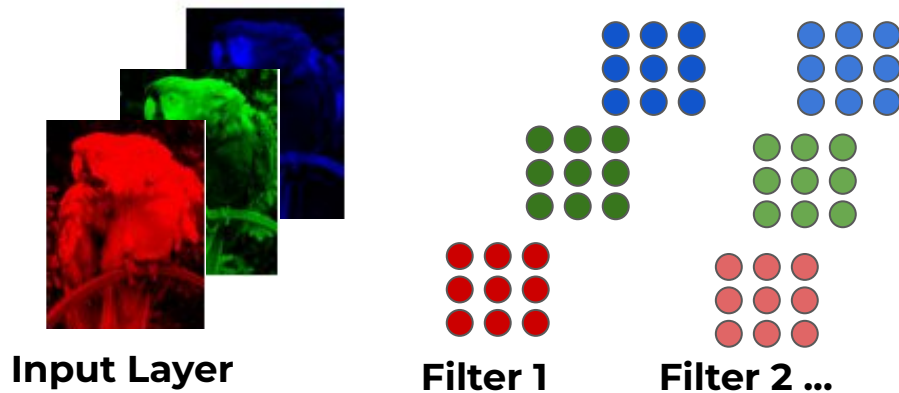


Filter





Deep Learning





Deep Learning

- Souvent, les couches de convolutions sont introduites dans une autre couche de convolution.
- Cela permet aux réseaux de découvrir des modèles à l'intérieur des modèles, généralement avec plus de complexité pour les couches de convolution ultérieures.





Deep Learning

- Nous avons beaucoup appris jusqu'à présent et nous avons un dernier sujet théorique à couvrir avant de coder nos propres CNNs, et c'est la couche de mise en pooling ! (également connue sous le nom de couche downsampling)



Couches de Pooling



MonCoachData



Deep Learning

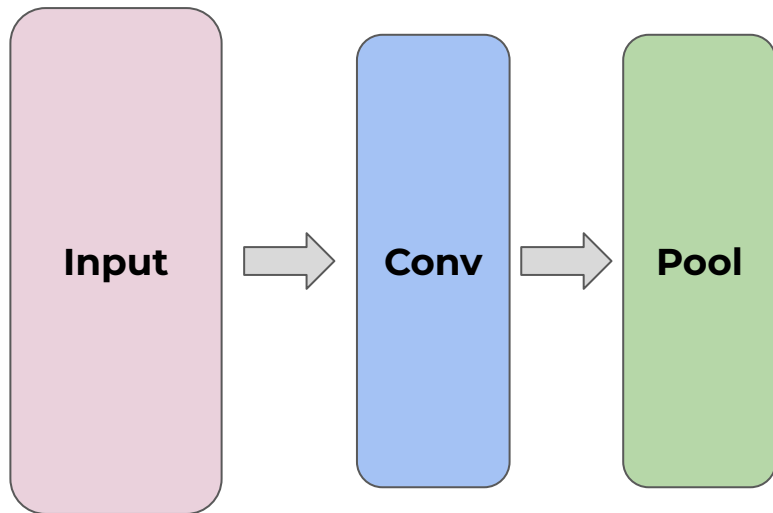
- Même avec la connectivité locale, lorsqu'il s'agit d'images en couleur et éventuellement de dizaines ou de centaines de filtres, nous aurons une grande quantité de paramètres.
- Nous pouvons utiliser des couches de Pooling pour réduire ce phénomène.





Deep Learning

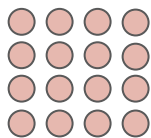
- Les couches de Pooling acceptent les couches convolutives comme entrée :





Deep Learning

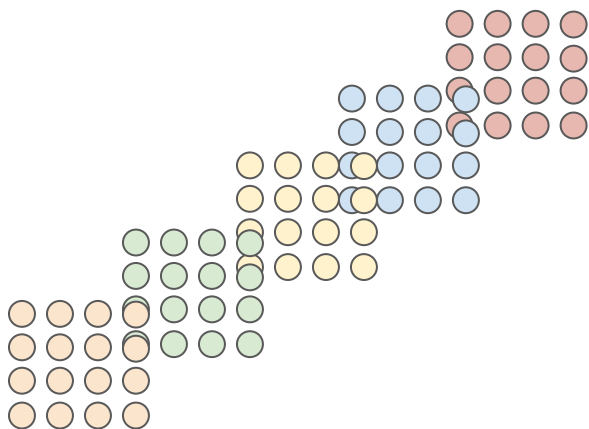
- Nos couches convolutives auront souvent de nombreux filtres





Deep Learning

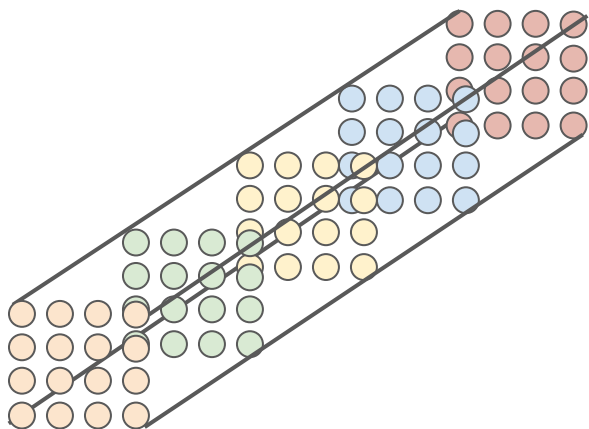
- Nos couches convolutives auront souvent de nombreux filtres





Deep Learning

- Nos couches convolutives auront souvent de nombreux filtres





Deep Learning

- Imaginons un filtre

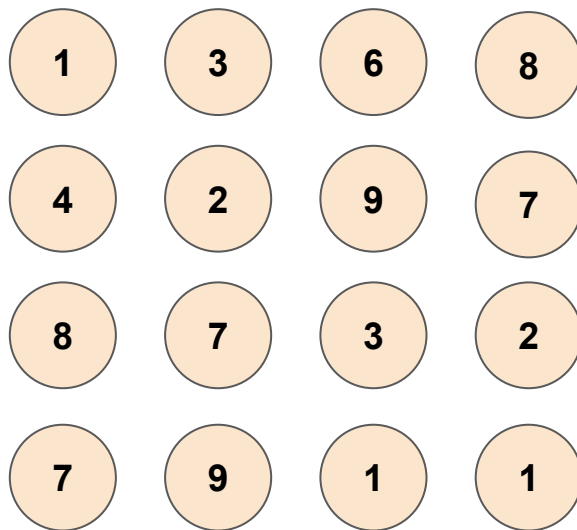
1	3	6	8
4	2	9	7
8	7	3	2
7	9	1	1





Deep Learning

- Nous pouvons réduire la taille avec le sous-échantillonnage





Deep Learning

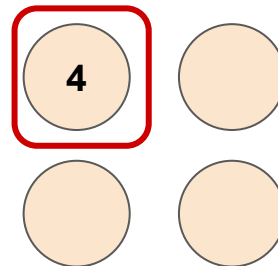
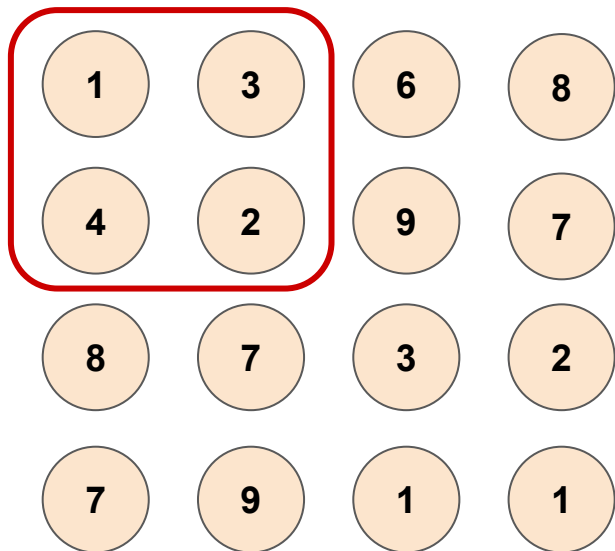
- Nous pouvons réduire la taille avec le sous-échantillonnage

1	3	6	8
4	2	9	7
8	7	3	2
7	9	1	1



Deep Learning

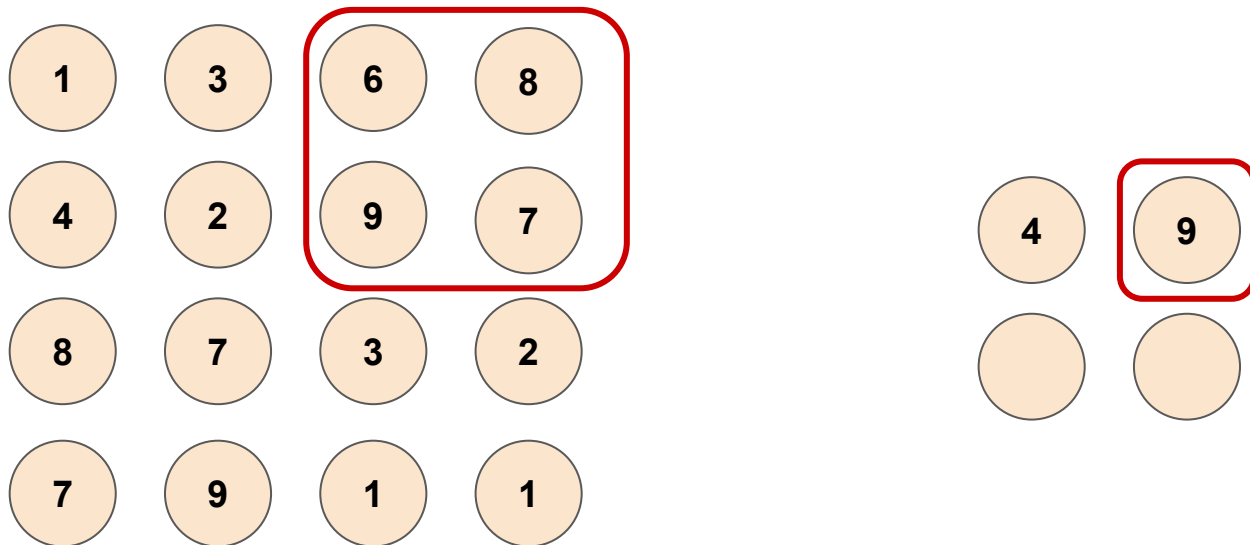
- Max Pooling: Window 2x2 , Stride: 2





Deep Learning

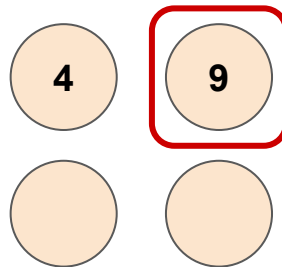
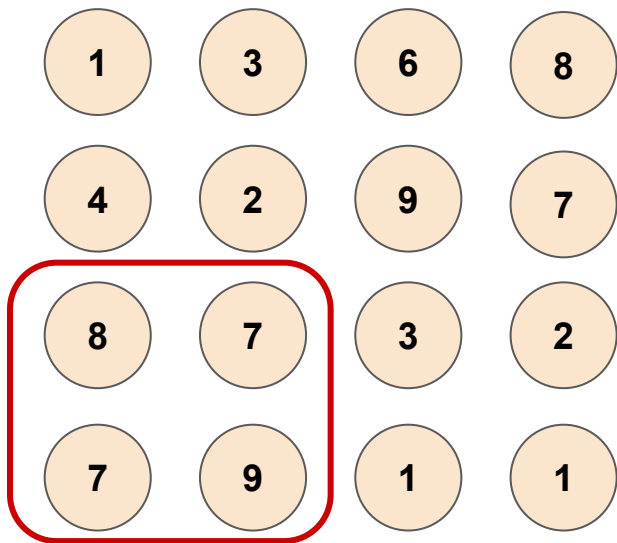
- Max Pooling: Window 2x2 , Stride: 2





Deep Learning

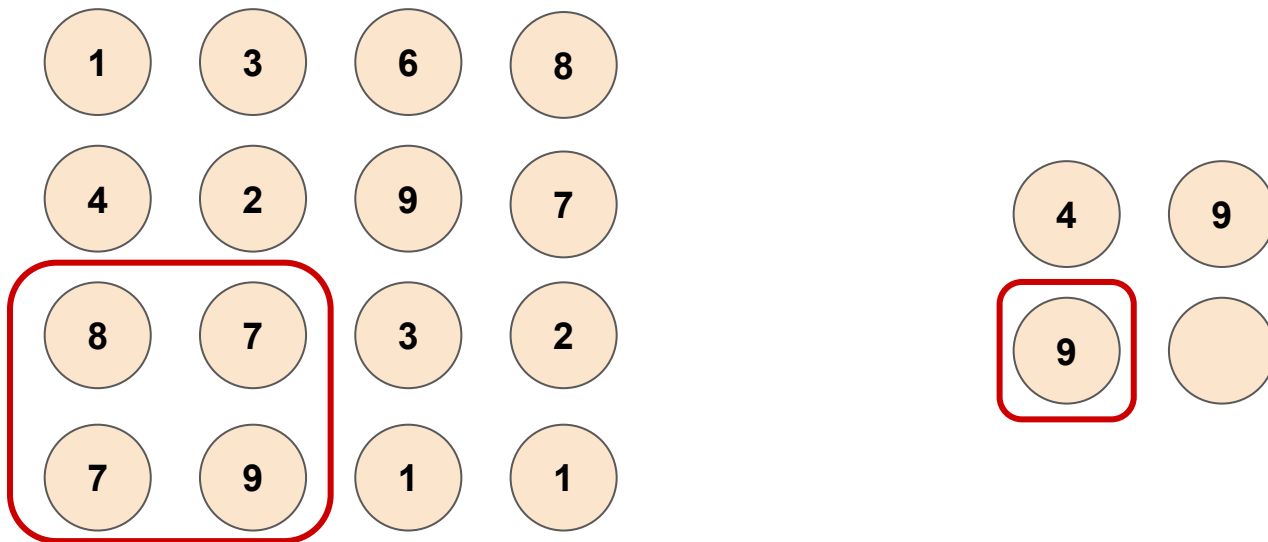
- Max Pooling: Window 2x2 , Stride: 2





Deep Learning

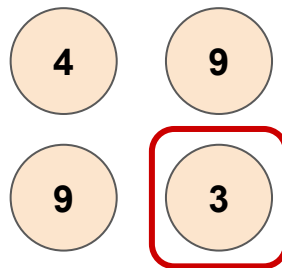
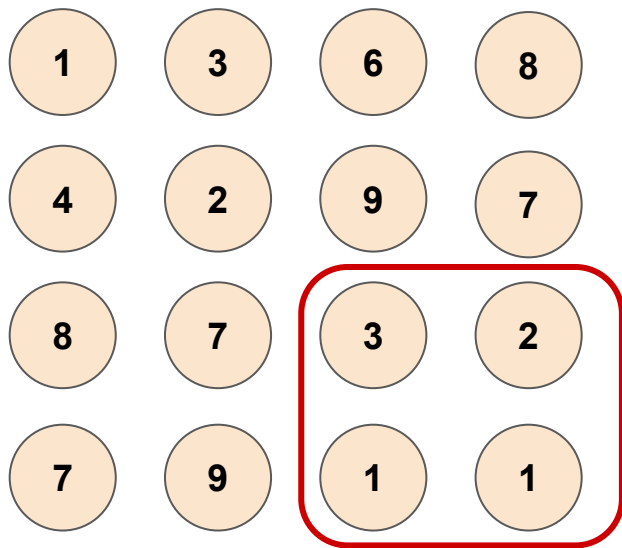
- Max Pooling: Window 2x2 , Stride: 2





Deep Learning

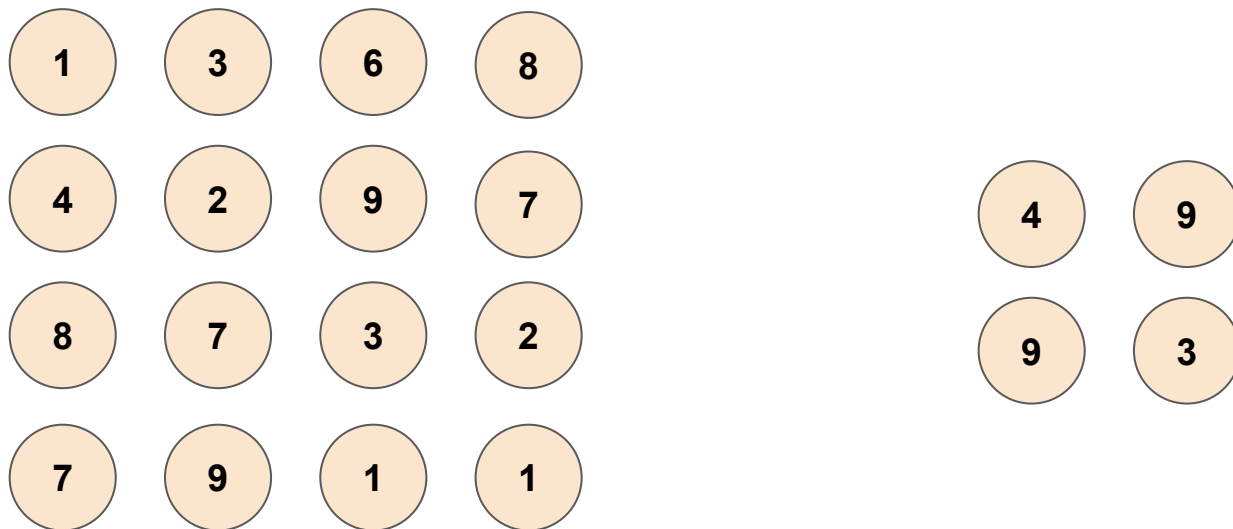
- Max Pooling: Window 2x2 , Stride: 2





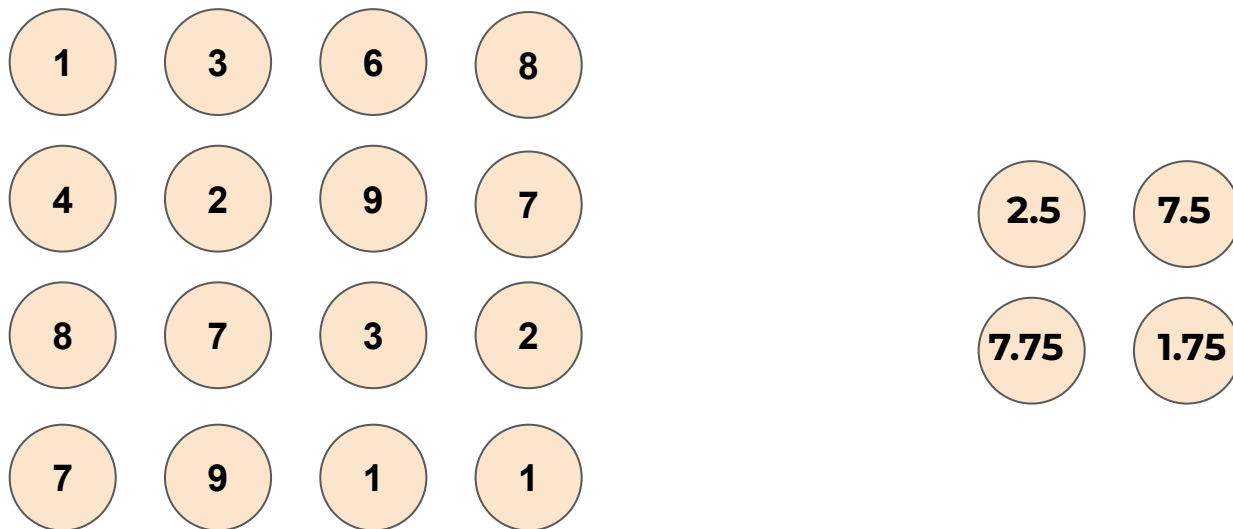
Deep Learning

- Max Pooling: Window 2x2 , Stride: 2





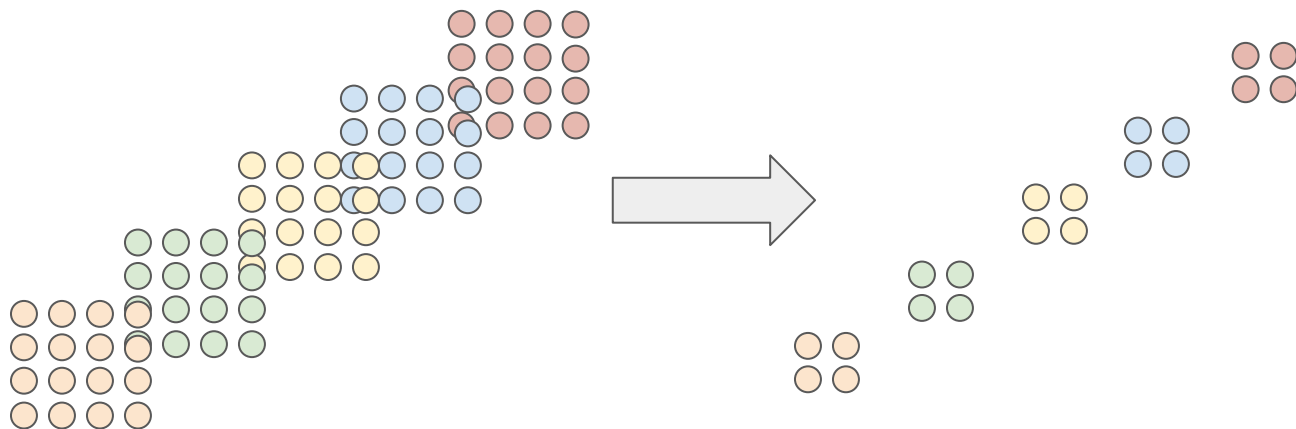
- Average Pooling: Window 2x2 , Stride: 2





Deep Learning

- Cela réduit considérablement le nombre de nos paramètres !





Deep Learning

- Cette couche de Pooling finira par supprimer beaucoup d'informations, même un petit "noyau" de Pooling de 2 par 2 avec un stride de 2 supprimera 75% des données d'entrée.



Deep Learning

- Une autre technique courante déployée avec un CNN est appelée "Dropout".
- Le Dropout peut être considéré comme une forme de régularisation qui permet d'éviter l'overfitting.
- Pendant l'entraînement, les neurones sont abandonnés au hasard, ainsi que leurs connexions.





Deep Learning

- Cela permet d'éviter que les unités ne "co-adaptent" trop.
- Montrons aussi rapidement quelques architectures célèbres de CNN !





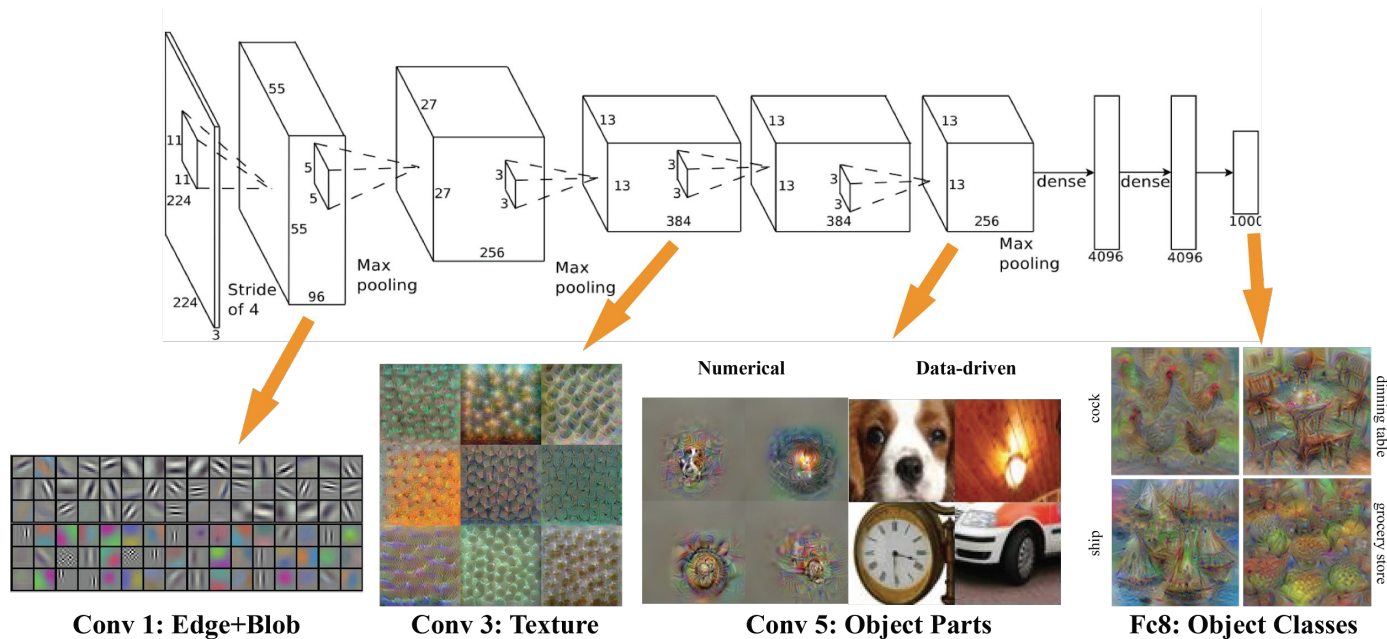
Deep Learning

- LeNet-5 par Yann LeCun
- AlexNet par Alex Krizhevsky et al.
- GoogLeNet par Szegedy à Google Research
- ResNet par Kaiming He et al.
- Consultez les liens vers les documents traitant de ces architectures !



Deep Learning

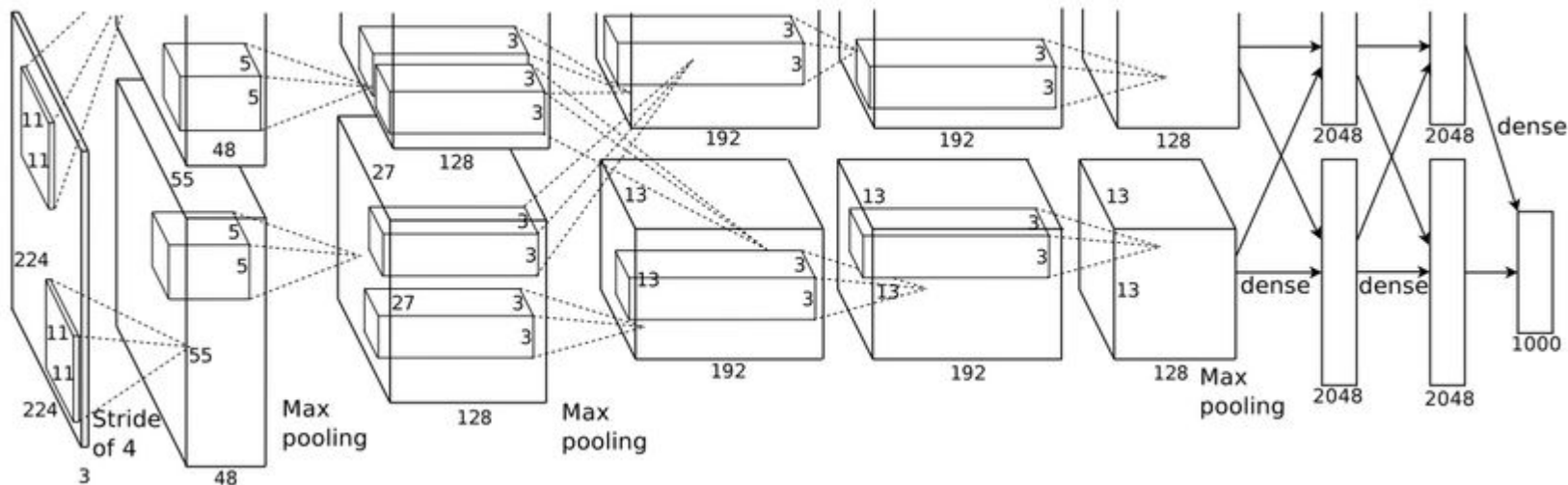
- AlexNet





Deep Learning

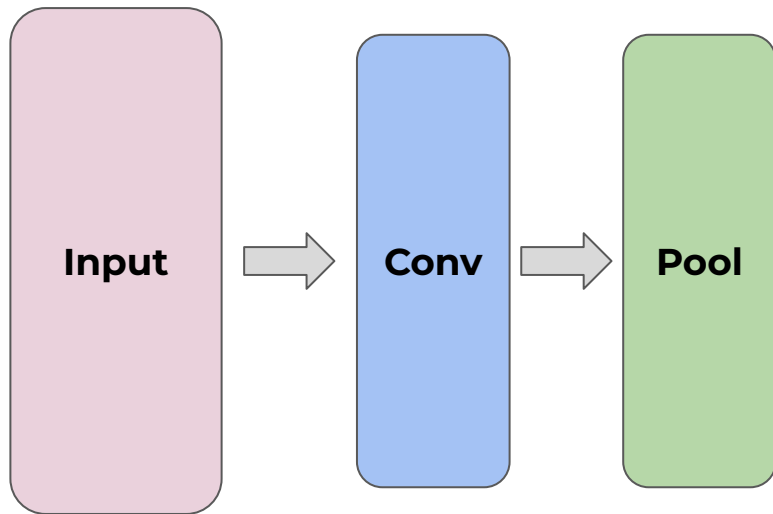
- AlexNet





Deep Learning

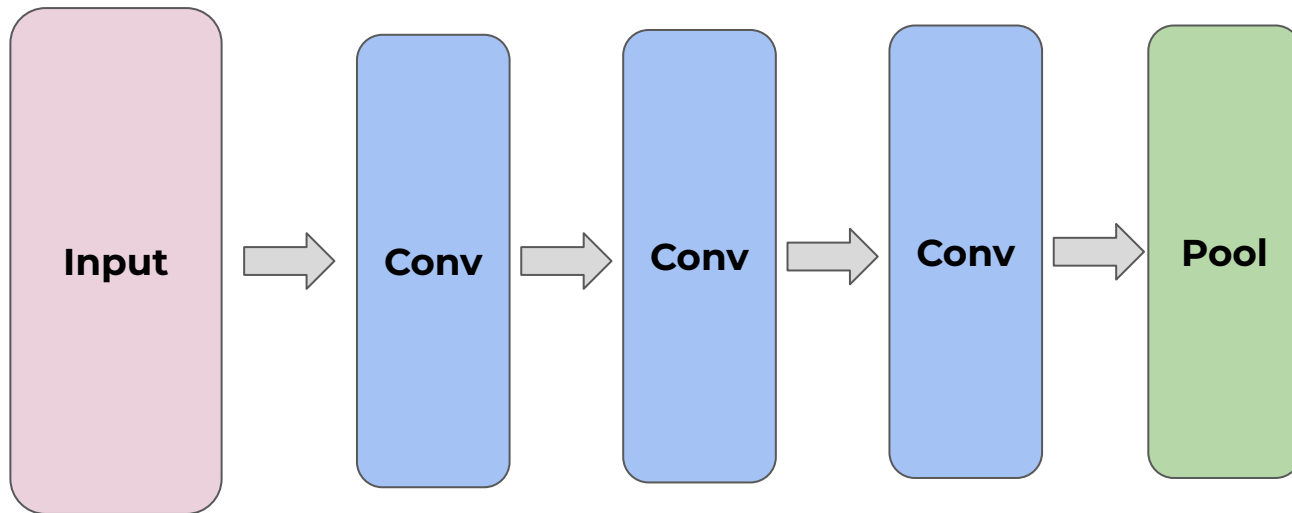
- Les CNNs peuvent avoir tous types d'architectures !





Deep Learning

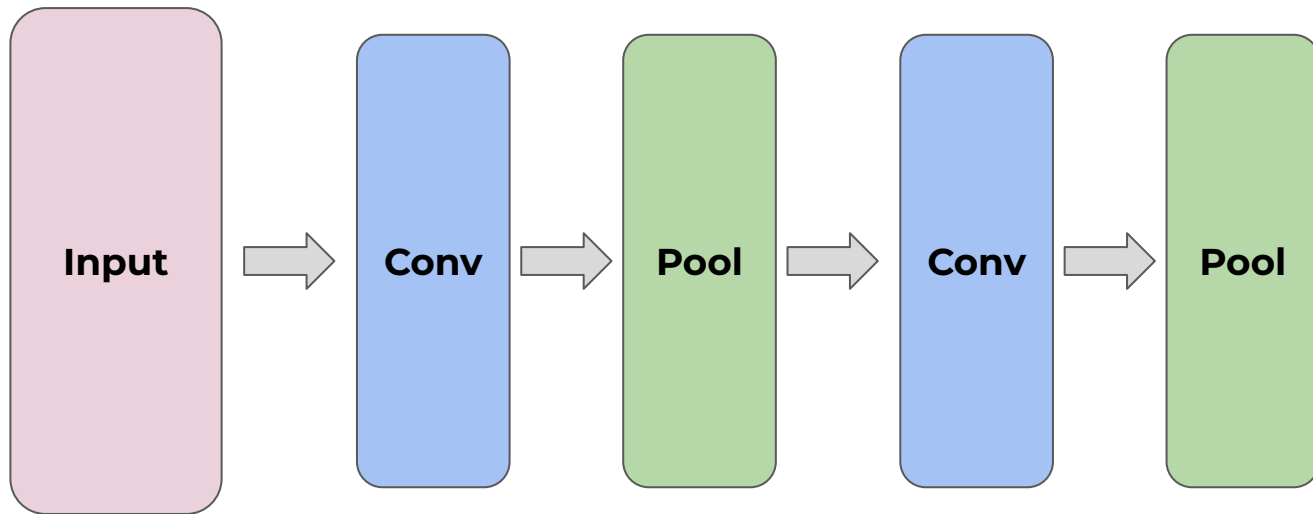
- Les CNNs peuvent avoir tous types d'architectures !





Deep Learning

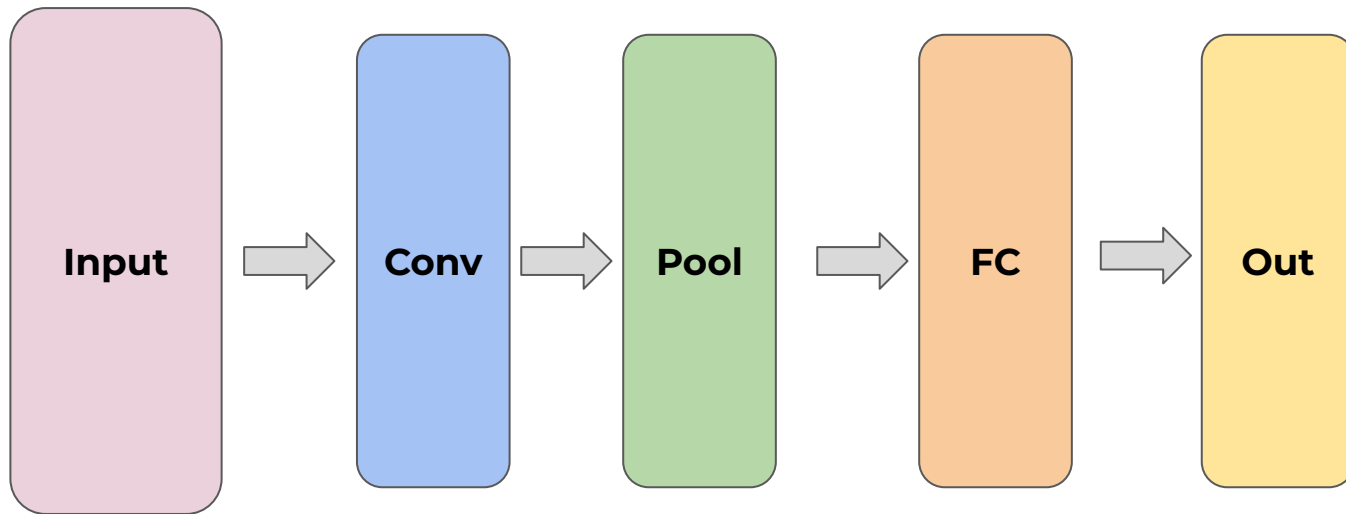
- Les CNN peuvent avoir tous types d'architectures !





Deep Learning

- Les CNN peuvent avoir tous types d'architectures !



Données MNIST



MonCoachData





Deep Learning

- L'ensemble de données MNIST est un ensemble de données classique dans le domaine du Deep Learning.
- Passons rapidement à l'essentiel puisque nous l'utiliserons assez fréquemment au cours de cette partie du cours !





Deep Learning

- Heureusement, ces données sont faciles d'accès avec Keras. Nous allons les télécharger :
 - 60,000 images d'entraînement
 - 10,000 images de test





Deep Learning

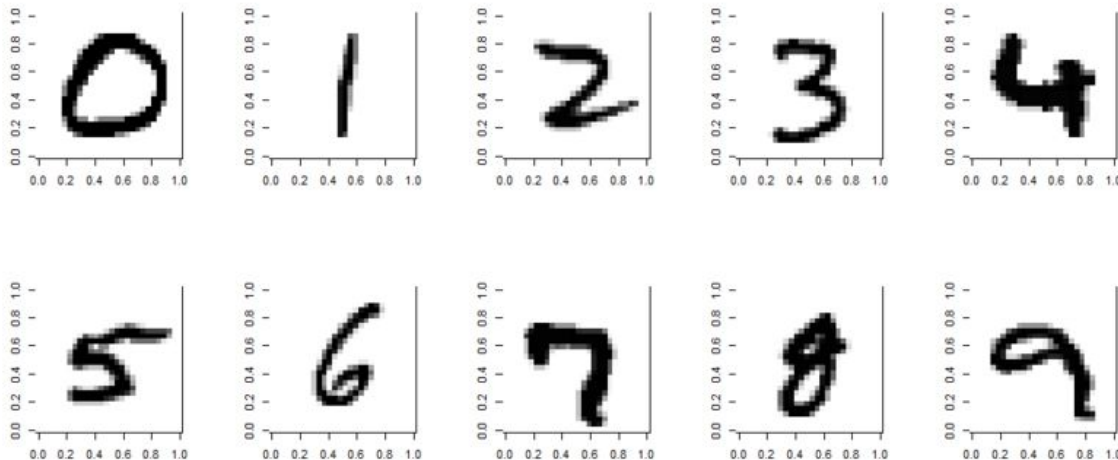
- L'ensemble de données MNIST contient des chiffres manuscrits uniques de 0 à 9





Deep Learning

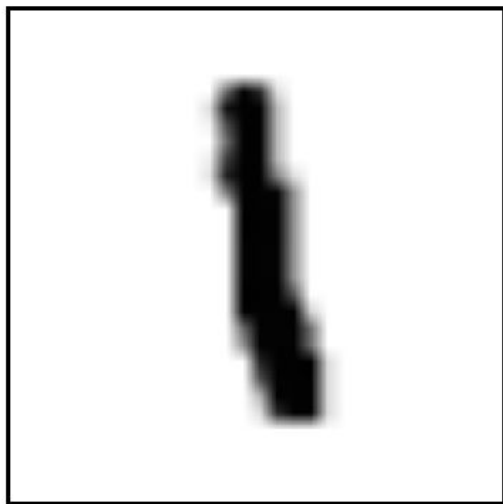
- Une image à un seul chiffre peut être représentée sous forme de tableau





Deep Learning

- Plus précisément, 28 par 28 pixels



21

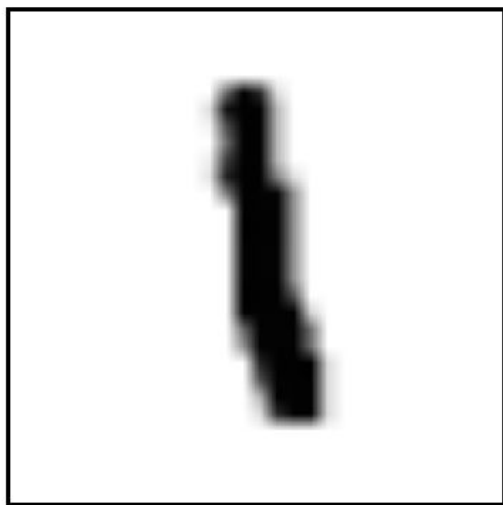
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	.6	.8	0	0	0	0	0	0
0	0	0	0	0	0	0	.7	1	0	0	0	0	0	0
0	0	0	0	0	0	0	.7	1	0	0	0	0	0	0
0	0	0	0	0	0	0	.5	1	.4	0	0	0	0	0
0	0	0	0	0	0	0	0	1	.4	0	0	0	0	0
0	0	0	0	0	0	0	0	1	.4	0	0	0	0	0
0	0	0	0	0	0	0	0	1	.7	0	0	0	0	0
0	0	0	0	0	0	0	0	1	1	0	0	0	0	0
0	0	0	0	0	0	0	0	.9	1	.1	0	0	0	0
0	0	0	0	0	0	0	0	.3	1	.1	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0





Deep Learning

- Les valeurs représentent l'image en niveaux de gris



12

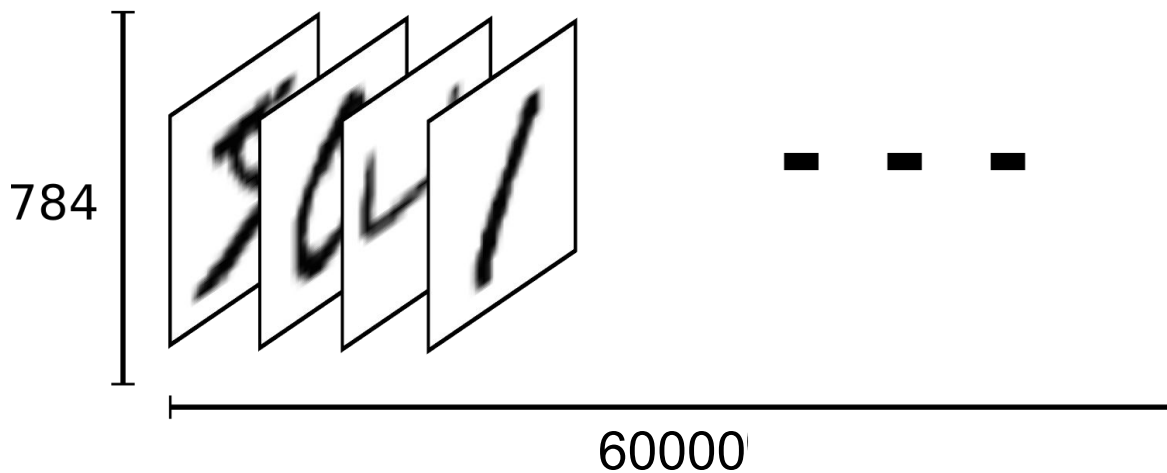
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	.6	.8	0	0	0	0	0	0
0	0	0	0	0	0	0	.7	1	0	0	0	0	0	0
0	0	0	0	0	0	0	.7	1	0	0	0	0	0	0
0	0	0	0	0	0	0	.5	1	.4	0	0	0	0	0
0	0	0	0	0	0	0	0	1	.4	0	0	0	0	0
0	0	0	0	0	0	0	0	1	.4	0	0	0	0	0
0	0	0	0	0	0	0	0	1	.7	0	0	0	0	0
0	0	0	0	0	0	0	0	1	1	0	0	0	0	0
0	0	0	0	0	0	0	0	.9	1	.1	0	0	0	0
0	0	0	0	0	0	0	0	.3	1	.1	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0





Deep Learning

- On peut considérer le groupe entier des 60 000 images comme un tableau en 4-dimensions.
60000 images de 1 canal 28 pixels par 28.





Deep Learning

- Ce tableau a **4** dimensions
 - (60000,28,28,1)
 - (Samples,x,y,channels)
- Pour des images en couleur, la dernière valeur de dimension serait 3 (RGB)





Deep Learning

- Pour les labels, nous utiliserons le One-Hot Encoding.
- Cela signifie qu'au lieu d'avoir des labels tels que "Un", "Deux", etc... nous aurons un seul tableau pour chaque image.





Deep Learning

- Cela signifie que les labels d'origine des images sont donnés en tant que liste de nombres
 - [5,0,4,...,5,6,8]
- Nous allons les convertir en one-hot encoding (facile avec Keras)





Deep Learning

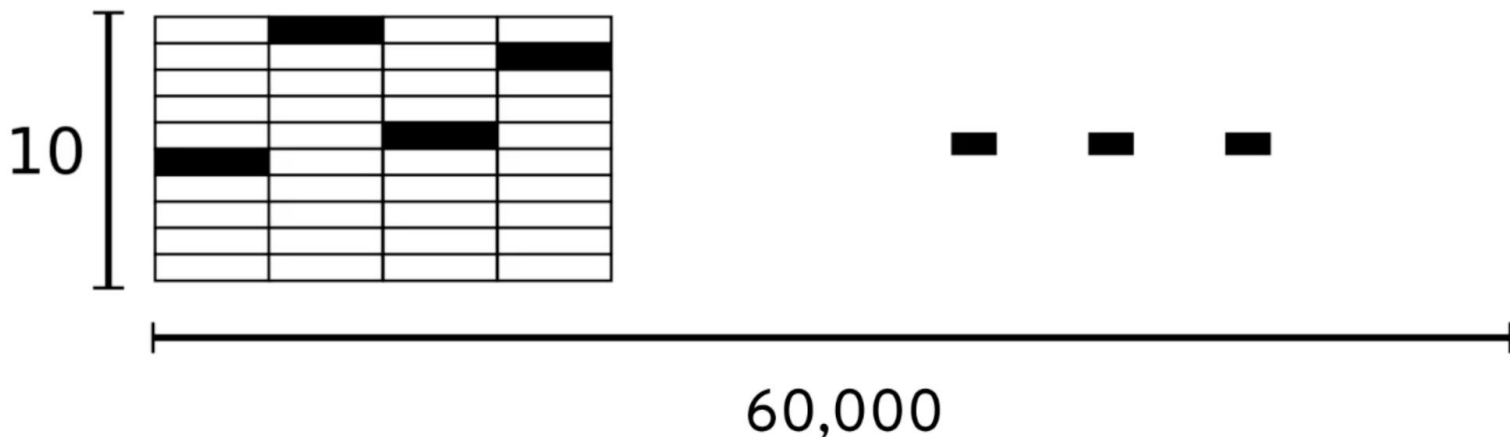
- Le label est représenté en fonction de la position de l'index dans le tableau des labels.
- Le label correspondant sera un 1 à l'emplacement de l'index et un 0 partout ailleurs.
- Par exemple, 4 aurait ce tableau de label :
 - `[0,0,0,0,1,0,0,0,0,0]`





Deep Learning

- En résultat, les labels pour les données d'entraînement finissent par être un large tableau en 2-d (60000,10) :





Deep Learning

- En résultat, les labels pour les données d'entraînement finissent par être un large tableau en 2-d (60000,10) :

```
y_train
```

```
array([[0., 0., 0., ..., 0., 0., 0.],  
       [1., 0., 0., ..., 0., 0., 0.],  
       [0., 0., 0., ..., 0., 0., 0.],  
       ...,  
       [0., 0., 0., ..., 0., 0., 0.],  
       [0., 0., 0., ..., 0., 0., 0.],  
       [0., 0., 0., ..., 0., 1., 0.]],
```



Deep Learning

- Maintenant que nous comprenons bien le dataset MNIST et l'encodage one-hot, appliquons nos nouvelles connaissances aux CNNs avec Keras sur ce dataset !



CNN avec MNIST

PARTIE 1 - DATA



MonCoachData



CNN avec MNIST

PARTIE 2 - MODÈLE ET ENTRAÎNEMENT



MonCoachData



CNN avec MNIST

PARTIE 3 - ÉVALUATION DU MODÈLE



MonCoachData



CNN avec CIFAR-10

PARTIE 1 - DATA



MonCoachData





Deep Learning

- Le dataset CIFAR-10 sont des images 32x32 de 10 objets différents :
 - Avion, Voiture, Oiseau, Chat, Cerf, Chien, Grenouille, Cheval, Bateau, Camion
 - Ce sont des images en couleur !





Deep Learning

- Dans cette série de vidéos, nous ré-utiliserons une grande partie du code CNN précédent, nous nous concentrerons donc uniquement sur les nouveaux ajouts dus à l'introduction de 3 canaux de couleur (RGB).



CNN avec CIFAR-10

PARTIE 2 - ÉVALUATION DU MODÈLE



MonCoachData



Téléchargement Des Données Images





Deep Learning

- Jusqu'à présent, nous n'avons traité que d'ensembles de données pré-conditionnés tels que le MNIST et le CIFAR-10.
- Mais qu'en est-il du travail avec des fichiers d'images réelles ? Comme les fichiers .jpg ou .png ?





Deep Learning

- Découvrons les outils intégrés de TensorFlow pour générer des batchs de données d'images à partir de répertoires contenant de vrais fichiers.
- Nous devons d'abord télécharger un gros fichier zip de données.





Deep Learning

- Cet ensemble de données est disponible sous la forme d'un fichier zip appelé **cell_images.zip**
- Ce fichier est disponible directement depuis le site web moncoachdata.com (le lien est présent dans les ressources de cette vidéo)
- Voyons comment obtenir ces données directement à partir de code Python !



CNN sur des images réelles

PARTIE 1 - DATA



MonCoachData



CNN sur des images réelles

PARTIE 2 - ImageDataGenerator



MonCoachData



CNN sur des images réelles

PARTIE 3 - CRÉATION DU MODÈLE



MonCoachData



CNN sur des images réelles

PARTIE 4 - ÉVALUATION DU MODÈLE



MonCoachData



Exercice CNN



Solution Exercise CNN

